

Answer Key — Final Exam SS 2025 (13 August 2025)

This is a reconstructed key, not an official one. ETH published no solutions for this paper. Every number below was recomputed in Python and every prose answer was checked against the lecture slides (cited as L_xx + slide number). Where the slides are genuinely ambiguous or the 2026 deck no longer covers the topic, it says so — those are the only places you should distrust this document.

Grading: 120 points. Pass = 60. Award partial credit generously for a correct formula with wrong arithmetic — the paper says “write down all intermediate steps”, and the real graders do the same.

Q	Topic	Max
1	Multiple Choice	16
2	Affective Computing	13
3	Deep Learning	13
4	Speech Recognition	13
5	Large Language Models	13
6	Speech Synthesis	13
7	Knowledge Graphs	13
8	Reinforcement Learning	13
9	Animation	13

Q1 — Multiple Choice (16 pts)

+1 correct, −1 wrong, 0 blank, block floors at 0.

	Statement (abbreviated)	Answer	Why
a	Artifact detection essential because EEG/ECG contaminated by noise	True	L_02 s.53. Muscle movement and environmental interference corrupt the signal.
b	Backprop gradients computed <i>independently</i> of activation functions	False	The chain rule multiplies $\sigma'(z)$ at every layer — the activation is inside the gradient.
c	DFT applied to short-time frames to track frequency over time	True	L_04. Speech is non-stationary.
d	Prosody refers <i>exclusively</i> to spectral features, no rhythm/intonation	False	L_08 s.26: prosody = prominence/accent, boundaries, duration, F_0 .
e	PEFT (LoRA, adapters) freezes most weights, learns few added parameters	True	L_05 s.70–72.
f	Prompt engineering controls tone/style/persona without modifying the model	True	L_06 s.4: system prompts set persona and constraints.
g	Hybrid fusion <i>eliminates</i> the need for modality-specific embedding	False	Each modality still gets its own encoder; fusion combines them, early <i>and</i> late.

	Statement (abbreviated)	Answer	Why
h	Stress depends on sentence context, accent is fixed within the word	False	Reversed. L_08 s.27: <i>accent</i> is a property of a word in context; <i>stress</i> is structural and fixed in the lexicon.
i	WaveNet uses causal dilated convolutions for long-range dependencies	True	L_09 s.27.
j	In LBS each vertex is influenced by exactly <i>one</i> bone	False	L_10. Each vertex is a weighted blend of <i>several</i> bones — that’s the “blend”.
k	Linear interpolation between keyframes often looks unnatural/abrupt	True	Disney’s “slow in and slow out” exists precisely because linear looks robotic.
l	Normalizing flows learn only the prior p_Z ; the transforms stay fixed	False	Backwards. The prior is a fixed simple $\mathcal{N}(0, 1)$; the invertible transforms are what’s learned.
m	An autonomous agent’s actions are determined <i>solely</i> by immediate perception	False	L_12 s.4: proactiveness — agents plan ahead and have control over internal states.
n	Dialogue trees have limited flexibility for unexpected/open-ended queries	True	L_12 s.11–13.
o	On-policy RL can reuse old-policy data from a replay buffer	False	That is the definition of off-policy . On-policy needs data from the current policy.
p	In Q-learning the policy is implicit: $a^* = \arg \max_a Q(s, a)$	True	L_14 s.29 (marked “for self-study” — still true).

Score: 16 if all correct. Count (correct) – (wrong), floor at 0.

NOTES

Q2 — Affective Computing (13 pts)**a) Two emotion theories from Fig. 1 (4 pts)**

One theory is missing from the figure. The three, in the lecture's terms (L_02):

- **James-Lange:** stimulus → **bodily/physiological response first** → the emotion is your *reading of* that bodily state. “I see the car, I tremble, therefore I am afraid.”
- **Cannon-Bard:** stimulus → physiological response **and** emotion fire **simultaneously and independently**. Seeing the car makes you tremble *and* feel fear at the same time; neither causes the other.
- **Schachter-Singer (two-factor):** stimulus → generic arousal → **cognitive interpretation of the context** → labelled emotion. Same racing heart is “fear” next to a car and “excitement” on a rollercoaster.

2 pts per case: name the theory + state the ordering that identifies it. The identification hinges entirely on **whether the arrow from the body to the emotion is sequential, parallel, or routed through a cognitive appraisal box**. That is what to look for in the figure.

b) “Anxious, tense, helpless” in the dimensional model (3 pts)

Dimension	Value	Reasoning (1 pt each)
Valence	Negative	Anxiety is an unpleasant state.
Arousal	High	Tension = physiological activation, elevated HR and skin conductance.
Dominance	Low	“Helpless” <i>is</i> the definition of low dominance — no control over the situation.

The word **helpless** is doing all the work in this question; it is there to force the dominance axis.

c i) Heart rate for the 400 ms R-R interval (1 pt)

$$\text{HR} = \frac{60000}{\text{RR}} = \frac{60000}{400} = \mathbf{150 \text{ bpm}}$$

c ii) Is 400 ms an artifact? (5 pts)

(1) **Sort** the intervals: [400, 795, 800, 800, 805, 810]

(2) **Quartiles and median** — lower half [400, 795, 800], upper half [800, 805, 810]:

$$Q_1 = 795, \quad Q_3 = 805, \quad \text{Median} = \frac{800 + 800}{2} = 800$$

$$(3) \text{ } QD = \frac{Q_3 - Q_1}{2} = \frac{805 - 795}{2} = \mathbf{5.00}$$

$$(4) \text{ } MAD = \frac{\text{Median} - 2.9 QD}{3} = \frac{800 - 14.5}{3} = \frac{785.5}{3} = \mathbf{261.83}$$

$$(5) \text{ } MED = 3.32 \times QD = 3.32 \times 5 = \mathbf{16.60}$$

$$(6) \text{ } CBD = \frac{MAD + MED}{2} = \frac{261.83 + 16.60}{2} = \mathbf{139.22}$$

(7) **Decision:** $|400 - 800| = 400 > 139.22 \Rightarrow$ **the 400 ms interval is an artifact.**

Worth a mark: 400 ms lies **inside** the 300–2000 ms validity window, so the simple range check would *not* have flagged it. The CBD test is what catches it. Say this explicitly.

Marking: 1 pt quartiles, 1 pt QD , 1 pt $MAD + MED$, 1 pt CBD , 1 pt the comparison and verdict.

NOTES

Q3 — Deep Learning (13 pts)

a i) Why RNNs suit sequential data (1 pt)

They carry a **hidden state** $\mathbf{h}_t = \sigma(U\mathbf{h}_{t-1} + W\mathbf{x}_t)$ forward from step to step, so the output at time t depends on all earlier inputs; and the same weights are **shared across time steps**, so the network handles variable-length sequences.

a ii) Why standard RNNs suffer vanishing gradients (1 pt)

Backpropagation through time multiplies the same Jacobian (and activation derivatives $\sigma' < 1$) once per time step. Over many steps the product shrinks **exponentially** toward zero, so gradients from distant time steps vanish and long-range dependencies are never learned.

a iii) LSTM gating (2 pts)

- **1 pt — the gates:** an LSTM adds a **cell state** c_t plus three gates: **forget** (what to erase from c_{t-1}), **input** (what new information to write), and **output** (what part of c_t to expose as h_t). Each gate is a sigmoid producing values in $[0, 1]$ that multiply elementwise.
- **1 pt — why it helps:** the cell state is updated **additively**, $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$, so gradients flow back along c through a near-identity path instead of a repeated multiplication. When the forget gate stays open, the gradient neither vanishes nor explodes.

b i) Positional encoding, positions 0 and 1 (3 pts)

The paper hands you the simplified forms $PE_{(pos,0)} = \sin(pos)$, $PE_{(pos,1)} = \cos(pos)$. **Radians.**

$$PE_0 = \begin{bmatrix} \sin 0 \\ \cos 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad PE_1 = \begin{bmatrix} \sin 1 \\ \cos 1 \end{bmatrix} = \begin{bmatrix} 0.84 \\ 0.54 \end{bmatrix}$$

Add (do **not** concatenate) to the embeddings:

$$\hat{\mathbf{x}}_0 = \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}, \quad \hat{\mathbf{x}}_1 = \begin{bmatrix} 3 \\ 4 \end{bmatrix} + \begin{bmatrix} 0.84 \\ 0.54 \end{bmatrix} = \begin{bmatrix} 3.84 \\ 4.54 \end{bmatrix}$$

$$\hat{X} = \begin{bmatrix} 1 & 3 \\ 3.84 & 4.54 \end{bmatrix}$$

Marking: 1 pt each PE , 1 pt the addition. A calculator left in degrees gives $\sin 1 = 0.017$ — costs the whole 3 points and poisons part ii.

b ii) Scaled dot-product attention (6 pts)

$Q = K = V = \hat{X}$, $d = 2$, so $\sqrt{d} = 1.414$.

Scores before softmax — $\hat{X}\hat{X}^\top$, then scaled:

$$\hat{X}\hat{X}^\top = \begin{bmatrix} 10.00 & 17.46 \\ 17.46 & 35.36 \end{bmatrix} \quad S = \frac{\hat{X}\hat{X}^\top}{\sqrt{2}} = \begin{bmatrix} 7.07 & 12.35 \\ 12.35 & 25.00 \end{bmatrix}$$

(Show both. “Scores before softmax” most naturally means the scaled matrix, but the unscaled one is the intermediate step and marking either alone as wrong would be harsh.)

Softmax, row by row. Subtract the row max before exponentiating — otherwise e^{25} overflows a pocket calculator:

- Row 1: gap = $12.35 - 7.07 = 5.28$, so $e^{-5.28} = 0.0051$ and $a_1 = \left[\frac{0.0051}{1.0051}, \frac{1}{1.0051}\right] = [0.005, 0.995]$
- Row 2: gap = $25.00 - 12.35 = 12.65$, so $e^{-12.65} = 3.2 \times 10^{-6}$ and $a_2 = [0.000, 1.000]$

$$A = \begin{bmatrix} 0.005 & 0.995 \\ 0.000 & 1.000 \end{bmatrix}, \quad \text{each row sums to } 1 \checkmark$$

Interpretation (this is a marked sub-point, don’t skip it): both tokens put essentially all their attention on **token 2**. Token 2’s embedding $[3.84, 4.54]$ has a much larger magnitude than token 1’s $[1, 3]$, so its dot

product with any query dominates, and the exponential in the softmax turns a moderate score gap into near-total dominance. Token 1 barely attends to itself at all.

Output = $A\hat{X}$:

$$\text{row 1} = 0.005 \begin{bmatrix} 1 \\ 3 \end{bmatrix} + 0.995 \begin{bmatrix} 3.84 \\ 4.54 \end{bmatrix} = \begin{bmatrix} 3.83 \\ 4.53 \end{bmatrix}, \quad \text{row 2} = \begin{bmatrix} 3.84 \\ 4.54 \end{bmatrix}$$

$$\text{Output} = \begin{bmatrix} 3.83 & 4.53 \\ 3.84 & 4.54 \end{bmatrix}$$

Both output rows collapse onto (almost exactly) token 2's value vector — the expected consequence of near-one-hot attention weights. **Marking:** 2 pts scores, 1 pt softmax, 1 pt interpretation, 2 pts output.

NOTES

Q4 — Speech Recognition (13 pts)

a i) Substitutions, insertions, deletions (3 pts)

- Reference: the quick brown fox jumps over the lazy dog — $N = 9$
- Output: the quick brown fax jump over lazy dogs — 8 words

Minimum-edit alignment:

Reference	Output	Edit
the	the	✓
quick	quick	✓
brown	brown	✓
fox	fax	S
jumps	jump	S
over	over	✓
the	—	D
lazy	lazy	✓
dog	dogs	S

$$S = 3, \quad I = 0, \quad D = 1$$

Sanity check that costs 5 seconds: $\text{len}(\text{hyp}) - \text{len}(\text{ref}) = 8 - 9 = -1 = I - D$. ✓

Trap: jump vs jumps and dog vs dogs are **substitutions**, not matches. Differing by an inflection is still the wrong word.

a ii) WER (2 pts)

$$\text{WER} = \frac{S + I + D}{N} = \frac{3 + 0 + 1}{9} = \frac{4}{9} = 0.444 = \mathbf{44.44\%}$$

1 pt for the formula, 1 pt for the number. Divide by the **reference** length, never the output length.

a iii) Theoretical min and max WER (2 pts)

- **Minimum** = 0 — a perfect transcription: no substitutions, insertions or deletions.
- **Maximum is unbounded** — it can exceed 100%. S and D are each capped by N , but I is **not**: the recognizer can emit arbitrarily many spurious words while the denominator stays fixed at the reference length.

Saying “maximum is 100 %” loses the point. That is the whole reason this sub-question exists.

b i) Power spectrum (4 pts)

$$P(k) = \frac{|X(k)|^2}{N}, \quad N = 3$$

$$P = \left[\frac{0^2}{3}, \frac{1^2}{3}, \frac{5^2}{3} \right] = \left[\frac{0}{3}, \frac{1}{3}, \frac{25}{3} \right] = [\mathbf{0}, \mathbf{0.33}, \mathbf{8.33}]$$

Meaning (2 of the 4 pts): the power spectrum gives the **energy at each frequency bin** of the frame — how much of each frequency component is present. Here almost all energy sits in bin $k = 2$, so that frequency dominates the frame.

b ii) Whole signal or short-time frames? (2 pts)

Short-time frames (about 20–40 ms, usually Hamming-windowed and overlapping). Speech is **non-stationary**: its frequency content changes as the speaker moves from phoneme to phoneme. A DFT over the entire utterance would average every phoneme together and destroy exactly the time-varying information that recognition depends on. Within a 20–40 ms window the signal is approximately stationary.

Q5 — Large Language Models (13 pts)

a) Prompt engineering vs fine-tuning (3 pts)

- **Prompt engineering (1 pt):** steer a **frozen** model purely through the input text — system prompt, instructions, in-context examples. No weight updates; effective immediately; costs nothing but context length.
- **Fine-tuning (1 pt):** continue training on task-specific data so the **weights themselves** change. Stronger and more consistent on the target task, but needs labelled data, compute, and a stored model copy per task.
- **The mechanism difference (1 pt):** fine-tuning changes the **parameters**; prompting changes only the **input**. Everything else — cost, latency, reversibility — follows from that one distinction.

b i) top-k vs top-p (2 pts)

- **top-k:** always keep a **fixed number** k of the highest-probability tokens, then sample.
- **top-p (nucleus):** keep the **smallest set** $V^{(p)}$ whose cumulative probability $\sum_{t \in V^{(p)}} P(t) \geq p$, then sample.
- **The difference that earns the mark:** top-p's candidate set is **dynamic** — it shrinks on a peaked distribution and grows on a flat one, so it adapts to the model's confidence. Top-k's is fixed regardless of shape, which either truncates a genuinely flat distribution or admits garbage from a peaked one.

b ii) Top-p with $p = 0.85$ (2 pts)

Token	$P(t)$	Cumulative
cat	0.35	0.35
dog	0.30	0.65
runs	0.15	0.80
fast	0.10	$0.90 \geq 0.85 \rightarrow \text{stop}$
tree	0.06	discarded
blue	0.04	discarded

Candidate set = {cat, dog, runs, fast}.

Then renormalize — the survivors sum to 0.90, not 1:

$$P'(t) = \frac{P(t)}{0.90} \Rightarrow [0.389, 0.333, 0.167, 0.111]$$

Trap: you **include** the token that crosses the threshold. Stopping at **runs** (cum. 0.80) because adding **fast** overshoots 0.85 is the classic wrong answer — $0.80 < 0.85$, so the condition isn't met yet.

c i) Cosine similarity (4 pts)

$$\cos(\mathbf{e}_A, \mathbf{e}_B) = \frac{\mathbf{e}_A \cdot \mathbf{e}_B}{\|\mathbf{e}_A\| \|\mathbf{e}_B\|}$$

$$\mathbf{e}_A \cdot \mathbf{e}_B = (0.5)(0.4) + (0.1)(0.3) + (0.4)(0.1) = 0.20 + 0.03 + 0.04 = 0.27$$

$$\|\mathbf{e}_A\| = \sqrt{0.25 + 0.01 + 0.16} = \sqrt{0.42} = 0.648$$

$$\|\mathbf{e}_B\| = \sqrt{0.16 + 0.09 + 0.01} = \sqrt{0.26} = 0.510$$

$$\cos = \frac{0.27}{0.648 \times 0.510} = \frac{0.27}{0.3305} = \mathbf{0.82}$$

Marking: 1 pt formula, 1 pt dot product, 1 pt the two norms, 1 pt result. A high value (0.82) means the two sentence embeddings point in a similar direction despite different lengths.

c ii) Perplexity of Sentence A (2 pts)

$$\text{PPL} = \left(\prod_{i=1}^N p_i \right)^{-1/N}, \quad N = 4$$

$$\prod_i p_i = 0.20 \times 0.10 \times 0.15 \times 0.25 = 0.00075$$

$$\text{PPL} = 0.00075^{-1/4} = \mathbf{6.04}$$

Trap: the exponent is **negative** $1/N$. Lower perplexity = better model.

NOTES

Q6 — Speech Synthesis (13 pts)

a i) Homograph disambiguation (2 pts)

Homographs are words spelled identically but pronounced differently depending on meaning or part of speech, so text normalization/phonetic analysis cannot map spelling to phonemes one-to-one.

Example (1 pt): “Do you *live* /l ih v/ near a zoo with *live* /l ay v/ animals?” Others: **present** (noun/verb), **read** (present/past), **bass**, **lead**.

How a TTS system resolves it (1 pt): run **part-of-speech tagging** on the sentence and look the word up in a **homograph dictionary** keyed by POS — verb **live** → /l ih v/, adjective **live** → /l ay v/. Modern systems use a learned classifier over the surrounding context instead of a rule table.

a ii) Limitation of pronunciation dictionaries (2 pts)

Limitation (1 pt): a dictionary is a **finite word list** (L_08 s.22: CMU 127 K words, Unisyn 110 K) and has nothing to say about **out-of-vocabulary words** — and those are not rare: L_08 s.23 puts **names alone at ~20 % of tokens** in typical newswire (personal names, brands: **McArthur**, **IKEA**). Neologisms, loanwords and inflections are missing too.

How modern systems address it (1 pt): **grapheme-to-phoneme (G2P) conversion** — a learned letter-to-sound model that predicts pronunciation for any unseen string from a sliding window of letters (L_08 s.24), with separate classifiers for names, plus morphological decomposition (**Lucasville** = **Lucas** + **ville**) and rhyme analogy (**Plotsky** from **Trotsky**).

a iii) Two prosodic phenomena (2 pts)

Pick any two of the four in L_08 s.26, one point each — **name it and say what it does:**

1. **Prominence / accent** — which words and syllables are made salient. A **nuclear accent** is the *last* accent in a phrase and is heard as the most prominent, marking contrast or focus. Distinct from **stress**, which is fixed in the lexicon and only marks *where* an accent could land.
2. **Intonation / F_0 contour** — the pitch trajectory over the utterance; e.g. a final rise marks a yes/no question, a fall marks a statement.
3. **Phrasing / boundaries** — intonation-phrase breaks, which lengthen the final vowel, insert a pause, and drop F_0 across the phrase.
4. **Duration** — segment lengths, which vary with stress, position and phrase-final lengthening.

b i) Tacotron vs FastSpeech (2 pts)

Architectural difference (1 pt): Tacotron 2’s decoder is **autoregressive with attention** — it emits one mel frame at a time, each conditioned on the previous one. FastSpeech is **non-autoregressive**: a **duration predictor** plus **length regulator** upsamples the encoder states, and a feed-forward Transformer emits **all mel frames in parallel**.

Impact (1 pt):

- **Speed:** FastSpeech is dramatically faster — parallel generation instead of a sequential loop over hundreds of frames.
- **Quality/robustness:** Tacotron 2 achieves excellent naturalness when its attention aligns, but the attention can **fail catastrophically** — skipped or repeated words (L_09 s.11). FastSpeech’s explicit durations make it robust to that and give direct control over speaking rate, at the cost of needing duration supervision and (in FastSpeech 1) slightly flatter prosody.

b ii) DiffWave (3 pts)

- **Forward process (1 pt):** a **fixed, non-learned** Markov chain that adds small amounts of Gaussian noise to the clean waveform over T steps, $x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_T$, until x_T is indistinguishable from pure $\mathcal{N}(0, I)$ noise. No training involved — it only generates (noisy input, noise) pairs to train on.
- **Reverse process (1 pt):** the **learned** direction. A neural network (a WaveNet-style stack of dilated convolutions) is trained to predict the noise added at each step, and generation starts from pure noise and **iteratively denoises** $x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_0$ to produce the waveform.
- **Conditioning (1 pt):** the reverse network is conditioned on the **mel-spectrogram** produced by the acoustic model (plus a diffusion-step embedding telling it how noisy the current input is). That is what makes it a *vocoder* rather than an unconditional audio generator.

b iii) Learned speaker embeddings over one-hot labels (2 pts)

Why prefer them (1 pt): one-hot labels enumerate a **closed set** of speakers fixed at training time — you cannot represent anyone outside the list, and adding a speaker means retraining (L_09 s.37). A learned embedding is a **continuous vector space** trained on a large speaker-labelled corpus, so similar voices land near each other, the space generalizes, and — if the training data is diverse enough — **zero-shot synthesis for a brand-new speaker from a single utterance** becomes possible (L_09 s.38). It also interpolates: you can blend between voices.

How to integrate (1 pt): train a separate **speaker-encoder** (CNN or RNN) on speaker-labelled audio, **freeze** it, then at both training and inference time run a reference utterance through it and feed the resulting embedding into the acoustic model — concatenated to the encoder output at every time step (or added as a conditioning bias to the decoder), exactly where the one-hot vector would have gone.

NOTES

Q7 — Knowledge Graphs (13 pts)

a) Two more triples + two relationship types (4 pts)

The two missing triples (1 pt each) — the paragraph states both facts and the schema licenses both:

(Dr. Sarah Clark, supervise, Mark Thompson)

(Mark Thompson, affiliatedWith, Computer Science Department)

“Dr. Clark supervises graduate student Mark Thompson, who is also affiliated with the same department.”

Two distinct types of relationship (1 pt each). The lecture’s taxonomy is L_12 s.43 — answer in those terms, not with a vague “there are people and papers”:

- **Asymmetric:** supervise. If Clark supervises Thompson, Thompson does **not** supervise Clark. Likewise published, lead, presentedAt.
- **1-to-N:** affiliatedWith. One department has many affiliated researchers — both Clark and Thompson point to the Computer Science Department.
- (Also defensible: **composition** — affiliatedWith followed by partOf composes into an implicit “researcher works at university” relation, Clark → CS Dept → ETH Zurich.)

Any two of these, correctly named and justified with an example from *this* graph, earns the marks.

b i) Query type (1 pt)

A conjunctive query. Three conditions are AND-ed together on the same variable (?researcher): affiliated with the CS Department, **and** published a paper presented at ICML, **and** leads a project. (One relation hop alone would be a *one-hop* query; a chain like Clark → CS Dept → ETH would be a *path* query.)

b ii) SPARQL (3 pts)

```
SELECT ?researcher
WHERE {
    ?researcher uni:affiliatedWith "Computer Science Department" .
    ?researcher uni:published      ?paper .
    ?paper      uni:presentedAt    "ICML" .
    ?researcher uni:lead          ?project .
}
```

Returns: Dr. Sarah Clark. (Mark Thompson satisfies the first condition only.)

Marking: 1 pt correct SELECT/WHERE skeleton, 1 pt all four triple patterns present, 1 pt for expressing the AND by **reusing the same ?researcher variable** across patterns — that is the actual concept being tested. Every line ends with ..

c i) TransE score for (Clark, supervise, Thompson) (2 pts)

Scoring function (1 pt — the paper says “specify” it, so write it down):

$$\text{score}(h, r, t) = \|\mathbf{h} + \mathbf{r} - \mathbf{t}\|_2 = \sqrt{\sum_i (h_i + r_i - t_i)^2}$$

Lower is better: a true triple should satisfy $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$.

Computation (1 pt):

$$\begin{bmatrix} 2.0 \\ 1.5 \\ -0.5 \end{bmatrix} + \begin{bmatrix} -0.2 \\ -0.7 \\ 0.3 \end{bmatrix} - \begin{bmatrix} 1.8 \\ 0.8 \\ -0.2 \end{bmatrix} = \begin{bmatrix} 1.8 - 1.8 \\ 0.8 - 0.8 \\ -0.2 - (-0.2) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\text{score} = \sqrt{0 + 0 + 0} = \mathbf{0.0}$$

A perfectly modelled triple — the translation lands exactly on the tail. Say that; it is the point of the question.

c ii) Is (Thompson, affiliatedWith, CS Dept) likely true? (1 pt)

$$\begin{bmatrix} 1.8 \\ 0.8 \\ -0.2 \end{bmatrix} + \begin{bmatrix} 1.2 \\ 0.6 \\ 0.8 \end{bmatrix} - \begin{bmatrix} 3.2 \\ 2.1 \\ 0.3 \end{bmatrix} = \begin{bmatrix} -0.2 \\ -0.7 \\ 0.3 \end{bmatrix}$$

$$\text{score} = \sqrt{0.04 + 0.49 + 0.09} = \sqrt{0.62} = \mathbf{0.79}$$

$0.79 < 2.0$, so the triple is **likely true** — consistent with the paragraph, which says Thompson is affiliated with the same department.

Trap — don't forget the square root. Stopping at 0.62 is the *squared* norm. Here both give the same verdict, but on a different paper they would not: for a residual $[-1.2, 1.2, -1.7]$ the sum of squares is 5.77 (above a threshold of 2) while the actual score is $\sqrt{5.77} = 2.40$ — same side here, but only by luck. Pick a norm, state it, stay consistent.

c iii) Why TransE fails on symmetric relations (2 pts)

The mathematics (1 pt). If both (h, r, t) and (t, r, h) are true, TransE requires

$$\mathbf{h} + \mathbf{r} \approx \mathbf{t} \quad \text{and} \quad \mathbf{t} + \mathbf{r} \approx \mathbf{h}$$

Add the two equations: $\mathbf{h} + \mathbf{t} + 2\mathbf{r} \approx \mathbf{t} + \mathbf{h}$, hence

$$2\mathbf{r} \approx \mathbf{0} \implies \mathbf{r} \approx \mathbf{0}$$

Substituting back into either equation gives $\mathbf{h} \approx \mathbf{t}$.

Implication for the learned embeddings (1 pt): the relation vector is forced to **collapse to zero**, so it carries no information and scores *every* pair of identical entities equally well; and the two entity embeddings are forced to become **identical**, so Dr. Clark and Mark Thompson become indistinguishable in the embedding space, destroying every *other* fact that needed to tell them apart. TransE therefore cannot represent symmetric relations at all (L_12 s.47) — which is what motivates TransR, where a relation-specific projection M_r can map two distinct entities onto the same point in relation space without merging them in entity space.

NOTES

Q8 — Reinforcement Learning (13 pts)

Setting: robot at $x_0 \in \mathbb{R}^2$, target $y_g \in \mathbb{R}^2$, dynamics $x_t = f(x_{t-1}, z_t)$.

a) State, action, policy (4 pts)

- **State s_t (1–2 pts):** what the agent perceives, and it must be **relevant to the goal** (L_14 s.50). Here: the robot’s current position **and the goal**, e.g. $s_t = (x_t, y_g)$ or equivalently $s_t = (x_t, y_g - x_t)$ — the position plus the displacement to the target. Adding velocity $\dot{x}_t$ is fine and arguably better. **A state of x_t alone is wrong:** without goal information the policy cannot know where to go.
- **Action a_t (1 pt):** the **control signal** $a_t = z_t$, since that is what the dynamics $f(x_{t-1}, z_t)$ consumes. **Continuous** action space, $z_t \in \mathbb{R}^2$.
- **Policy (1–2 pts):** $\pi : \mathcal{S} \rightarrow \mathcal{A}$, mapping the state to an action. Deterministic: $a_t = \mu(s_t)$. Stochastic (usual choice here): $a_t \sim \pi(\cdot | s_t)$, where the network outputs the **parameters of a distribution** — e.g. mean and variance of a Gaussian — that you sample from. Because the action space is continuous, the network is a **regressor**, not a classifier (L_14 s.14).

b) Reward r_t and return $R(\tau)$ (2 pts)

Reward (1 pt) — “reach the target **as quickly as possible**” means the reward must encode *both* goal progress and time. Either of these earns it:

$$r_t = -\|x_t - y_g\| \quad \text{or} \quad r_t = \|x_{t-1} - y_g\| - \|x_t - y_g\| - c$$

i.e. reward the reduction in distance to the goal, minus a **constant time penalty c per step** (or equivalently a +1 bonus on reaching the goal and -1 per elapsed step). A pure sparse “+1 if $x_T = y_g$ ” is acceptable but worth mentioning as hard to learn.

Return (1 pt) — defined over the **whole trajectory**, with a discount factor (L_14 s.17):

$$R(\tau) = \sum_{t=0}^T \gamma^t r_t, \quad \gamma \in (0, 1]$$

The discount is what makes “as quickly as possible” fall out: a reward received later is worth less, so the optimal policy reaches the goal early.

c) $Q(s, a_n)$ and $V(s)$ (4 pts)

State-action value function (2 pts). Q is the expected return **after committing to action a_n in state s** , then following the policy. In this finite-sample setup with N sampled actions each rolled out for T steps:

$$Q(s, a_n) = \mathbb{E}_{\tau \sim \pi}[R(\tau) | s_0 = s, a_0 = a_n] \approx \sum_{t=0}^T \gamma^t r_{t,n}$$

that is, the discounted return of trajectory τ_n .

State value function (2 pts). V is the expected return from s **before** the action is chosen — so it is Q averaged over the policy’s action distribution (L_14 s.21):

$$V(s) = \mathbb{E}_{a \sim \pi(\cdot | s)}[Q(s, a)] \approx \sum_{n=1}^N \pi(a_n | s) Q(s, a_n)$$

If the N actions are drawn **from** π (as stated), the Monte-Carlo estimate simplifies to the plain average $V(s) \approx \frac{1}{N} \sum_{n=1}^N Q(s, a_n)$ — the sampling already applies the weighting. **Either form earns full marks; showing you know why they coincide is the strongest answer.**

The relationship in one line: Q fixes the first action, V averages over it. Their difference is the **advantage** $A(s, a) = Q(s, a) - V(s)$ — which is exactly what part d) is about.

d) Three advantages of PPO + Actor-Critic over vanilla policy gradient (3 pts)

One point each. The vanilla estimator is $\nabla_{\theta} J = \mathbb{E}[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) R(\tau)]$.

1. **Lower variance → stable training.** Vanilla policy gradient weights every action by the *whole* trajectory return, which has enormous variance and makes training unstable and prone to divergence (L_14 s.35–36). The critic supplies a **state-dependent baseline** $V(s)$ and the gradient uses the **advantage** $A^{\pi}(s_t, a_t) = Q(s_t, a_t) - V(s_t)$ instead of $R(\tau)$. Because the baseline depends only on the state, it reduces variance **without introducing bias** (L_14 s.37).
2. **Controlled, stable policy updates.** A single bad gradient step can move the policy far enough to collapse training and ruin subsequent data collection (L_14 s.44). PPO optimizes a **clipped surrogate objective** on the probability ratio $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}$:

$$L(\theta, \theta_k) = \mathbb{E}[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

which removes the incentive to move the policy outside a trust region.

3. **Better sample efficiency.** Vanilla policy gradient must throw away each batch after **one** gradient step. The clipping makes it safe to take **multiple epochs of updates on the same batch**, so far more learning happens per unit of environment interaction — and the critic bootstraps Q from the temporal difference of V (L_14 s.42), so you don't need full Monte-Carlo rollouts to estimate returns.

Other defensible points: works with continuous action spaces; no need for a differentiable reward or environment; simpler to implement and tune than TRPO for the same trust-region benefit.

NOTES

Q9 — Animation (13 pts)

Three links $L = [2, 2, 1]$, all joints at $\theta_i^0 = \pi/6$ (30°), target $\mathbf{e}^* = [5, 2]^\top$, learning rate $\alpha = 0.1$. **Radians throughout.**

Cumulative angles — the only thing you actually need:

$$\theta_1 = 30^\circ, \quad \theta_1 + \theta_2 = 60^\circ, \quad \theta_1 + \theta_2 + \theta_3 = 90^\circ$$

$$\cos : 0.866, 0.5, 0 \quad \sin : 0.5, 0.866, 1$$

a i) Initial end-effector position (2 pts)

$$\mathbf{e}^0 = \begin{bmatrix} L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) + L_3 \cos(\theta_1 + \theta_2 + \theta_3) \\ L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2) + L_3 \sin(\theta_1 + \theta_2 + \theta_3) \end{bmatrix} = \begin{bmatrix} 2(0.866) + 2(0.5) + 1(0) \\ 2(0.5) + 2(0.866) + 1(1) \end{bmatrix} = \begin{bmatrix} \mathbf{2.73} \\ \mathbf{3.73} \end{bmatrix}$$

a ii) Error (1 pt)

$$\Delta \mathbf{e} = \mathbf{e}^* - \mathbf{e}^0 = \begin{bmatrix} 5 - 2.73 \\ 2 - 3.73 \end{bmatrix} = \begin{bmatrix} \mathbf{2.27} \\ \mathbf{-1.73} \end{bmatrix}$$

Target minus current, never the reverse. Getting the sign backwards moves the arm away from the ball and costs everything downstream.

a iii) Jacobian (3 pts)

$$J = \frac{\partial \mathbf{e}}{\partial \theta} = \begin{bmatrix} -L_1 s_1 - L_2 s_{12} - L_3 s_{123} & -L_2 s_{12} - L_3 s_{123} & -L_3 s_{123} \\ L_1 c_1 + L_2 c_{12} + L_3 c_{123} & L_2 c_{12} + L_3 c_{123} & L_3 c_{123} \end{bmatrix}$$

Column j drops every term before joint j — rotating joint 3 cannot move links 1 and 2. Row 1 is $\partial x / \partial \theta$ (sines, negated); row 2 is $\partial y / \partial \theta$ (cosines).

$$J = \begin{bmatrix} -2(0.5) - 2(0.866) - 1(1) & -2(0.866) - 1(1) & -1(1) \\ 2(0.866) + 2(0.5) + 1(0) & 2(0.5) + 1(0) & 1(0) \end{bmatrix} = \begin{bmatrix} \mathbf{-3.73} & \mathbf{-2.73} & \mathbf{-1.00} \\ \mathbf{2.73} & \mathbf{1.00} & \mathbf{0.00} \end{bmatrix}$$

Free check: row 2 of J equals the x -components of $\mathbf{e}^0$ accumulated from the right, and $J_{2,1} = 2.73 = e_x^0$. If your $J_{2,1}$ doesn't match the x you computed in (i), one of them is wrong.

a iv) Change in rotation angles (2 pts)

$$\Delta \theta = \alpha J^+ \Delta \mathbf{e}$$

With the J^+ printed on the paper:

$$J^+ \Delta \mathbf{e} = \begin{bmatrix} -0.29 & 0.47 \\ -0.19 & 0.19 \\ -0.05 & 0.12 \end{bmatrix} \begin{bmatrix} 2.27 \\ -1.73 \end{bmatrix} = \begin{bmatrix} (-0.29)(2.27) + (0.47)(-1.73) \\ (-0.19)(2.27) + (0.19)(-1.73) \\ (-0.05)(2.27) + (0.12)(-1.73) \end{bmatrix} = \begin{bmatrix} -0.658 - 0.813 \\ -0.431 - 0.329 \\ -0.114 - 0.208 \end{bmatrix} = \begin{bmatrix} -1.47 \\ -0.76 \\ -0.32 \end{bmatrix}$$

$$\Delta \theta = 0.1 \begin{bmatrix} -1.47 \\ -0.76 \\ -0.32 \end{bmatrix} = \begin{bmatrix} \mathbf{-0.15} \\ \mathbf{-0.08} \\ \mathbf{-0.03} \end{bmatrix}$$

Do not forget the α . It is the single most commonly dropped factor in this question.

a v) Update the angles (2 pts)

$$\theta^1 = \theta^0 + \Delta \theta = \begin{bmatrix} 0.52 \\ 0.52 \\ 0.52 \end{bmatrix} + \begin{bmatrix} -0.15 \\ -0.08 \\ -0.03 \end{bmatrix} = \begin{bmatrix} \mathbf{0.38} \\ \mathbf{0.45} \\ \mathbf{0.49} \end{bmatrix} \text{ rad}$$

($\pi/6 = 0.5236$.) Answer stays in **radians** — no conversion asked for. All three angles decreased, which is right: the ball is low and to the right, so the arm has to rotate clockwise (negative) to reach it.

b i) DeepPhase (1 pt)

A **periodic autoencoder** that learns a **phase manifold** from motion data: it extracts periodic phase variables from the joint-motion curves, so that cyclic motion such as walking is represented by where it sits in its cycle rather than by raw pose. Motions can then be aligned and blended **in phase space**, which removes the misalignment artifacts (foot sliding, averaged-out gaits) you get when blending naively in pose space.

b ii) Normalizing flows + one application (2 pts)

Main idea (1 pt): model a complex distribution as a chain of **invertible, differentiable** transformations applied to a simple prior $z \sim \mathcal{N}(0, I)$:

$$x = f(z) = f_1(f_2(\dots f_N(z))), \quad z = f_N^{-1}(\dots f_1^{-1}(x))$$

Because every step is invertible, you get **both** directions: sample z and push it forward to generate new data, **or** push a data point backward and evaluate its **exact** likelihood $p(x)$ via the change-of-variables formula (using the Jacobian determinant of each transform). That exact-density property is what distinguishes flows from GANs and VAEs.

Application (1 pt): speech-driven gesture synthesis — L_11 s.32–33, *Style-Controllable Speech-Driven Gesture Synthesis Using Normalizing Flows* (GestureFlow / MoGlow, [ALE20]), where affine coupling layers generate diverse, style-controllable co-speech gestures. Character motion synthesis / locomotion generation is equally acceptable.

NOTES

Self-grading

Add up the sub-question points you earned, then record the total in [progress.md](#). What matters is not the total but **which** questions bled points:

- **Lost points on a calculation?** That is the cheap fix. Re-drill it: `python3 drill.py <recipe>`.
- **Lost points on prose?** Check whether the answer is already in [DEFINITIONS.md](#). If it is, that's a memorization gap. If it isn't, tell me and I'll add it.
- **Lost points on the MC block?** Deck 00_Exam_MC_Traps covers these. Remember: a wrong answer costs -1 , a blank costs 0 . If you were guessing, you were losing.

Calculation points available on this paper: Q2c (6), Q3b (9), Q4a+b i (9), Q5b ii + c (8), Q7c (5), Q9a (10) = **47 points**, plus the 16-point MC block = **63**. That is the pass, without writing a single paragraph of prose.

NOTES
